

Electricity Bill Calculator

Evaluated Course Project Report

Course: CSE1021 — Problem Solving through
Python Course Type: Flipped Course
VIT Bhopal University

Submitted by: AASTIK DIXIT
Registration Number: 26BCE11483

Submission Date: 30 September 2026

1. Introduction

This report documents the design, implementation, and testing of the Electricity Bill Calculator, a command-line Python application built as the Evaluated Course Project for CSE1021 (Problem Solving through Python). The project applies core Python concepts — functions, conditional logic, loops, and modular program structure — to solve a practical, real-world problem: calculating electricity bills using tiered (slab) billing rates.

The project is organized into independent modules for billing calculation, input validation, and bill record-keeping, connected through a single entry point that drives a menu-based user interface.

2. Problem Statement

Manually calculating electricity bills using tiered (slab) rates is repetitive and error-prone when done by hand, especially as the number of customers grows. Small billing offices or clerks often need a fast, reliable way to compute a customer's bill without manually working out which slab each unit falls into. This project addresses that problem by automating slab-based bill calculation and providing a simple record of all bills generated in a session.

3. Functional Requirements

- 1. Bill Calculation Module** — Accept a customer name and units consumed, then calculate the energy charge using a 4-tier slab rate structure, plus a fixed charge, and display an itemised bill.
- 2. Input Validation Module** — Reject empty customer names and non-numeric or invalid unit values before any calculation is performed.
- 3. Bill Records Module** — Store every bill generated during the session and allow the user to view the full list of bills along with the total amount collected.

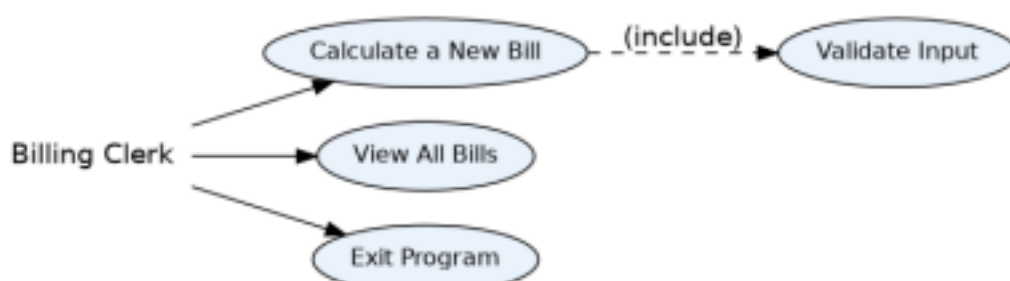
4. Non-functional Requirements

- 1. Usability** — a simple, numbered menu interface that requires no prior training to use.
- 2. Reliability** — the slab calculation logic is deterministic and produces consistent results for the same input, verified through automated tests.
- 3. Maintainability** — the codebase is split into independent modules (billing, validation, records), so each concern can be modified or extended without affecting the others.
- 4. Error Handling** — all user input is validated before being used in a calculation; invalid input produces a clear message instead of crashing the program.

5. System Architecture

The application follows a simple modular architecture. A single entry point (main.py) drives the menu-based user interface and delegates work to three independent modules: billing.py for slab rate calculations, validation.py for input checking, and records.py for storing and retrieving bill history. This separation keeps each module focused on a single responsibility and makes the system easy to test and extend

6. WORKFLOW



7. Design Decisions & Rationale

- 1. Modular file structure over a single script** — splitting the project into `billing.py`, `validation.py`, and `records.py` makes each piece of logic independently testable and easier to reason about, compared to one large script.
- 2. Function-based design over classes** — since the project scope does not require multiple independent objects with their own state, plain functions keep the code simple and readable, which fits the course's focus on procedural problem solving.
- 3. In-session storage using lists** — bill records are kept in memory using simple parallel lists rather than a file or database, since persistence between runs was not a required feature and this keeps the project dependency-free.
- 4. Slab rate constants defined at the top of `billing.py`** — this keeps the billing rules in one clearly visible place, so they can be updated without touching the calculation logic itself.

10. Implementation Details

The project is implemented in Python 3 using only the standard language features — no external libraries are required. The core billing logic in `billing.py` uses nested `if/elif/else` statements to apply the correct slab rate based on the number of units consumed. Input validation in `validation.py` uses simple string methods (`str.strip()` and `str.isdigit()`) to check for empty or non-numeric input before it reaches the calculation logic. Bill records are stored in `records.py` using three parallel lists (customer names, units used, and bill amounts), with helper functions to add a record and retrieve the full history and grand total. `main.py` ties these modules together behind a simple while-loop-driven menu.

Slab Rate Structure

UNITS CONSUMED	RATE PER UNIT
First 100 units	Rs 3
Next 100 units (101-200)	Rs 5
Next 100 units (201-300)	Rs 7
Above 300 units	Rs 10

A fixed charge of Rs 50 is added to every bill.

10. Screenshots / Results

The screenshot below shows a real run of the program: calculating a bill for 275 units.

```
-----ELECTRICITY BILL CALCULATOR-----
1. Calculate a new bill
2. View all bills
3. Exit
-----BILL-----
Customer      :  AASTIK DIXIT
Units used    :  275
Energy charge : Rs.  1325
Fixed charge  : Rs.   50
Total charge  : Rs.  1375
-----
```

11. Testing Approach

A dedicated test file, `test_project.py`, verifies the correctness of the billing and validation modules using Python's assert statements. The tests cover boundary values across all four slab tiers (50, 100, 150, 250, and 350 units) to confirm the slab logic switches correctly at each threshold, as well as valid and invalid name/unit inputs for the validation functions.

Manual testing was also performed by running the full program and exercising all three menu options (calculate a bill, view all bills, exit), including deliberately entering invalid input (empty names, non-numeric units) to confirm the error messages appear correctly.

All automated tests pass, confirmed by running: `python test_project.py`, which outputs ALL TESTS PASSED.

12. Challenges Faced

1. Correctly calculating the cumulative slab charges for units that span multiple tiers (e.g. 250 units spans three different rate tiers) required careful attention to make sure each tier's units were charged at the correct rate rather than applying one rate to the whole amount.
2. Deciding how to split a single working script into separate modules without breaking functionality meant thinking carefully about which pieces of logic belonged together (e.g. keeping all slab math inside `billing.py` rather than mixing it with the menu code).
3. Validating input safely — for example, making sure a blank or whitespace-only name was correctly rejected — required testing edge cases beyond just normal expected input.

13. Learnings & Key Takeaways

This project reinforced how splitting a program into small, focused modules makes code easier to read, test, and debug compared to one large script. Writing automated tests before making further changes also helped catch a calculation error early (an incorrect expected value in a test case) that would have been harder to notice through manual testing alone. The project also demonstrated how a real-world billing problem — tiered pricing — can be modeled cleanly using basic conditional

logic.

14. Future Enhancements

1. Persist bill records to a file (e.g. JSON or CSV) so history survives between program runs, rather than resetting each session.
2. Add the ability to search or filter bills by customer name.
3. Support configurable slab rates and fixed charges, loaded from a settings file instead of hard-coded constants.
4. Add a simple graphical or web interface for non-technical users.

15. References

1. Python Software Foundation, The Python Standard Library, docs.python.org
2. Python Software Foundation, Python 3 Language Reference, docs.python.org/3/reference
3. VITyarthi Course Materials — CSE1021: Problem Solving through Python